House Price Prediction with Linear Models

DSS5104 — CA 1

March 2026

1. Introduction

This report presents a comprehensive approach to predicting residential house prices using linear regression models. The primary objective is to demonstrate that, through careful and creative feature engineering, linear models can achieve competitive predictive performance while retaining the interpretability, transparency, and simplicity that make them invaluable in real-world deployment settings.

The dataset comprises 9,200 property listings from the state of Washington, USA, with 18 attributes including structural characteristics, location information, and sale prices. The target variable is the sale price, and model performance is evaluated using Mean Absolute Percentage Error (MAPE), which emphasizes relative prediction accuracy across the wide range of house prices observed in the data.

The modelling pipeline follows a structured approach: exploratory data analysis (EDA) to understand the data, systematic data cleaning, extensive feature engineering informed by domain knowledge, rigorous cross-validated model selection, and interpretation of results. A well-tuned XGBoost model serves as a non-linear benchmark to contextualize the performance of the linear approach.

2. Data Description and Exploratory Analysis

2.1 Dataset Overview

The raw dataset contains 9,200 rows and 18 columns. Each row represents a property sale record with the following attributes: sale date, price, number of bedrooms and bathrooms, living area and lot sizes (in square feet), number of floors, waterfront status, view rating, condition rating, above-ground and basement square footage, year built, year renovated, street address, city, state-zip code, and country.

All columns are fully populated with no missing values. The price column is numeric (float64), while structural attributes are a mix of integer and float types. Location fields (street, city, statezip, country) are categorical strings.

2.2 Key Findings from EDA

Duplicates: A significant portion of the dataset consists of exact duplicate rows: 4,598 out of 9,200 (49.98%). These were identified and removed during preprocessing, leaving 4,602 unique records.

Price Distribution: The price distribution is heavily right-skewed, with most properties priced between $200,000 and $800,000, but a long tail extending beyond $2 million. This skewness motivates the use of log-transformed prices as the modelling target.

Zero and Negative Prices: 98 rows were found with price values of zero, likely representing data entry errors or non-arm's length transactions.

Outliers: Analysis of price-per-square-foot identified extreme outliers: rows with price/sqft exceeding $5,000 or below $20, as well as 4 rows where both bedrooms and bathrooms were zero (physically implausible for a habitable property). In total, 5 extreme outlier rows were removed.

Location: All properties are in Washington state (country is uniformly “USA”). There are 44 distinct cities and 77 unique zip codes. Some cities have very few listings (fewer than 10), which required special handling during stratified splitting and encoding.

Dropped Columns: The “street” column has 4,525 unique values out of 9,200 rows, making it near-unique and thus not useful for generalizable modelling. The “country” column is constant (“USA”). The 'statezip' column is superseded by the extracted numeric zip code. The “date” column is converted into temporal features (month) before being dropped.

3. Data Cleaning and Preprocessing

The data cleaning pipeline proceeds as follows:

1. Deduplication: Removed 4,598 exact duplicate rows, reducing the dataset from 9,200 to 4,602 records.
2. Date parsing: Converted the date column to datetime format and extracted sale_month as temporal features.
3. Zip code extraction: Parsed numeric zip codes from the statezip field (e.g., “WA 98133” becomes “98133”).
4. Invalid price removal: Removed 49 rows with price at or below zero.
5. Extreme outlier removal: Removed 5 rows with implausible price-per-sqft values or zero bedrooms and bathrooms.
6. Column removal: Dropped “date”, “street”, “statezip”, and “country” columns.

After cleaning, the dataset contains 4,548 rows and 17 columns. The data was then split into training (3,638 rows, 80%) and test (910 rows, 20%) sets using stratified sampling by “city”, with rare cities (fewer than 10 listings) grouped as “Other” for stratification purposes.

4. Feature Engineering

Feature engineering is the centrepiece of this project. A total of 134 features were constructed from the original 17 cleaned columns. All feature engineering is encapsulated in a stateful “FeatureEngineer” class that is fit exclusively on training data and applied to test data, ensuring no data leakage. The features fall into the following categories:

4.1 Log Transformations

Given the right-skewed distributions of area-related variables, log transformations (log1p) were applied to “sqft_living”, “sqft_lot”, “sqft_above”, “sqft_basement”, and derived ratio features. These transformations help linearize the relationship between size and log-price, enabling the linear model to capture diminishing returns in property size.

4.2 Age and Renovation Features

House age is computed as (2014 – “yr_built”), reflecting the approximate age at the time of sale. An “effective_age” feature captures whether the property was renovated, using the renovation year when available and the original build year otherwise. Additional features include a binary “was_renovated” indicator, a log-transformed effective age, and an “age_decade” feature (house age binned by decade). These features capture the non-linear effect of property aging, where very old homes may command premiums as 'character' properties.

4.3 Size Ratios and Composite Features

Several ratio features were engineered to expose property characteristics that raw dimensions alone cannot convey: “sqft_per_room” (living area divided by total rooms), “basement_frac” (basement area as a proportion of total living area), “bed_bath_ratio” (bedrooms per bathroom), and “living_to_lot” (building footprint relative to lot size). Binary indicator features flag premium properties (waterfront, high view rating, or top condition), atypical properties (many/few bedrooms, very large/small houses, very old or newly built), and properties with basements.

4.4 Polynomial and Interaction Terms

To capture non-linear relationships within a linear framework, squared terms were created for key variables: “log_sqft_living”, “log_sqft_lot”, “log_effective_age”, “bathrooms”, “condition”, and “view”. Interaction terms model joint effects, such as “log_living_x_cond” (larger homes in better condition command disproportionate premiums), “log_living_x_view” (size amplifies the value of views), “wf_x_view” (waterfront combined with views), and “age_x_reno” (the value of renovation depends on age).

4.5 Seasonal Features

Cyclical encoding of the sale month using sine and cosine transformations (sin(2*pi*month/12) and cos(2*pi*month/12)) captures any seasonal patterns in pricing without imposing an artificial ordinal structure on months.

4.6 Location Encodings

Location is one of the strongest predictors of house price. Rather than one-hot encoding 44 cities and 77 zip codes (which would be sparse and high-dimensional), smoothed target encodings were used. For each city and zip code, the median log-price was computed on training data and smoothed toward the global mean using a Bayesian shrinkage formula with a smoothing parameter of m = 50. This prevents overfitting for categories with few observations. Additional location features include: the difference between zip-code and city-level encodings “zip_city_diff”, count-based encodings (how many listings appear in each city/zip code), and a zip-code price variability encoding “zip_std” capturing how heterogeneous prices are within a given area.

4.7 KNN Neighbourhood Features (Dual-Scale)

K-Nearest Neighbours was used purely as a feature construction tool (not as a prediction model). For each property, the 10 nearest and 25 nearest neighbours were identified in a feature space comprising size, rooms, floors, condition, view, age, and location encodings. Summary statistics of neighbours' log-prices were extracted: mean, median, standard deviation, min, max, percentiles (25th, 75th), IQR, and distance-weighted mean. A broader 25-neighbour version provides a second, more smoothed estimate. The difference between local (k=10) and broad (k=25) averages captures whether a property is priced above or below its broader neighbourhood trend. During training, self-exclusion ensures no leakage.

4.7 KMeans Clustering Features

KMeans clustering (k=8) was applied to a subset of structural features (living area, bedrooms, bathrooms, condition, floors, effective age) to create property-type clusters. One-hot encoded cluster memberships and the distance to each property's assigned cluster centre were added as features. These clusters capture latent property archetypes that may have distinct pricing dynamics.

4.9 Spline Basis Expansions

Spline transformers (cubic B-splines) were fitted on six key variables: “log_sqft_living”, “log_sqft_lot”, “house_age”, “zip_enc”, “city_enc”, and “effective_age”. These basis expansions allow the linear model to capture flexible non-linear patterns without abandoning the linear modelling framework. For instance, the relationship between house age and price is distinctly non-monotonic, and splines handle this naturally.

4.10 Standard Scaling

All 134 features are standardized (zero mean, unit variance) using a StandardScaler fitted on training data. This is essential for regularized linear models (Ridge, Lasso), where the penalty term is sensitive to feature scales.

5. Model Training and Selection

5.1 Target Transformation

The target variable is log-transformed (log1p(price)) during training. Predictions are mapped back to price space via expm1 before computing MAPE. This log transformation reduces the influence of extreme high-value properties and aligns with the multiplicative nature of housing markets, where factors tend to scale prices by percentages rather than absolute amounts.

5.2 Cross-Validation Strategy

Model selection uses 5-fold cross-validation with feature engineering re-fit inside each fold. This is critical: because target encodings and KNN features are derived from training labels, fitting them on the full training set before CV would leak information. By re-fitting the entire FeatureEngineer pipeline within each fold, the CV estimates remain unbiased.

5.3 Candidate Models and Hyperparameter Search

Thirteen candidate configurations were evaluated:

- Ridge regression with MAPE-balancing sample weights (alpha = 0.05, 0.1, 0.5, 1.0, 5.0, 10.0): six configurations. The MAPE-balancing weights (1/log1p(price)) down-weight expensive properties to ensure the model optimizes relative error across all price ranges.
- Lasso regression without sample weights (alpha = 1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3, 0.01): seven configurations. Lasso provides automatic feature selection through L1 regularization.

The following table summarises the cross-validation results for all candidates, sorted by mean CV MAPE:

Model	CV MAPE	CV Std
Ridge (alpha = 10.0, weighted)	0.1768	0.0160
Ridge (alpha = 5.0, weighted)	0.1769	0.0162
Lasso (alpha = 0.001)	0.1771	0.0163
Ridge (alpha = 1.0, weighted)	0.1773	0.0173
Lasso (alpha = 0.0005)	0.1774	0.0169
Ridge (alpha = 0.5, weighted)	0.1774	0.0179
Lasso (alpha = 0.01)	0.1801	0.0150

Table 1: Cross-validation results for linear model candidates (selected rows shown).

The best single model is Ridge regression with alpha = 10.0 and MAPE-balancing sample weights, achieving a CV MAPE of 0.1768. All 134 coefficients are non-zero, which is expected for Ridge (L2 regularization shrinks but does not eliminate coefficients). The performance gap between Ridge and Lasso variants is small, suggesting that the engineered features are generally informative and that aggressive feature elimination is not beneficial.

5.4 Stacked Linear Model

To further improve performance, a stacked ensemble of two linear models was constructed:

- Base learners: Ridge (alpha = 1.0) and Lasso (alpha = 0.001)
- Meta-learner: Ridge (alpha = 1.0) trained on out-of-fold predictions from the base learners

The stacking procedure uses 5-fold cross-validation to generate out-of-fold (OOF) predictions for the training set, while test-set predictions are averaged across folds. Feature engineering is re-fit within each fold to prevent leakage. The stacked model achieves an OOF MAPE of 0.1767, closely matching the best single-model CV estimate of 0.1768, which indicates consistent and reliable generalization.

5.5. XGBoost Benchmark

A well-tuned XGBoost model was trained as a non-linear benchmark. The hyperparameter search used 50 random configurations evaluated via 5-fold cross-validation, with feature engineering re-fit within each fold to prevent leakage. The optimal parameters were: 1,500 estimators, max depth of 4, learning rate of 0.01, subsample of 0.7, colsample_bytree of 0.6, min_child_weight of 1, reg_alpha of 0.1, reg_lambda of 0.5, and gamma of 0.0.

6. Results

6.1 Test Set Performance

The final test-set metrics for all three models are summarised below:

Metric	Single Ridge	Stacked Linear	XGBoost
MAPE	16.81%	16.74%	15.64%
MAE	$94,901	$93,972	$89,018
RMSE	$180,765	$183,677	$207,897
R^2	0.8026	0.7962	0.7390
R^2 (log)	0.8222	0.8238	0.8350

Table 2: Test-set performance comparison across all models.

The stacked linear model achieves a test MAPE of 16.74%, representing the best linear approach. The XGBoost benchmark achieves 15.64%, a gap of approximately 1.1 percentage points. Notably, the linear models outperform XGBoost on RMSE ($180,765 vs $207,897) and R^2 (0.80 vs 0.74), indicating that while XGBoost is slightly better at minimizing relative error, the linear models produce more consistent absolute-value predictions with fewer large outlier errors.

6.2 Performance by Price Segment

The segment-level MAPE analysis reveals where models succeed and struggle:

Segment	Count	Stacked MAPE
< $200K	34	46.9%
$200K–$400K	317	15.0%
$400K–$600K	283	15.9%
$600K–$800K	147	14.3%
$800K–$1M	57	14.7%
$1M–$2M	63	19.0%
> $2M	9	30.4%

Table 3: MAPE by price segment on the test set.

The model performs best in the $200K – $1M range (MAPE of 14 – 16%), which contains the bulk of the data (804 out of 910 test observations). Performance degrades at both extremes: properties below $200K (MAPE ~47%) and above $2M (MAPE ~30%). The very low-price segment likely contains atypical sales (distressed properties, partial interests), while ultra-luxury properties are inherently harder to predict due to idiosyncratic features not captured in the dataset.

6.3 Predicted vs. Actual Plots

Figure 1 shows the predicted-versus-actual scatter plot and residual plot for the stacked linear model on the test set.

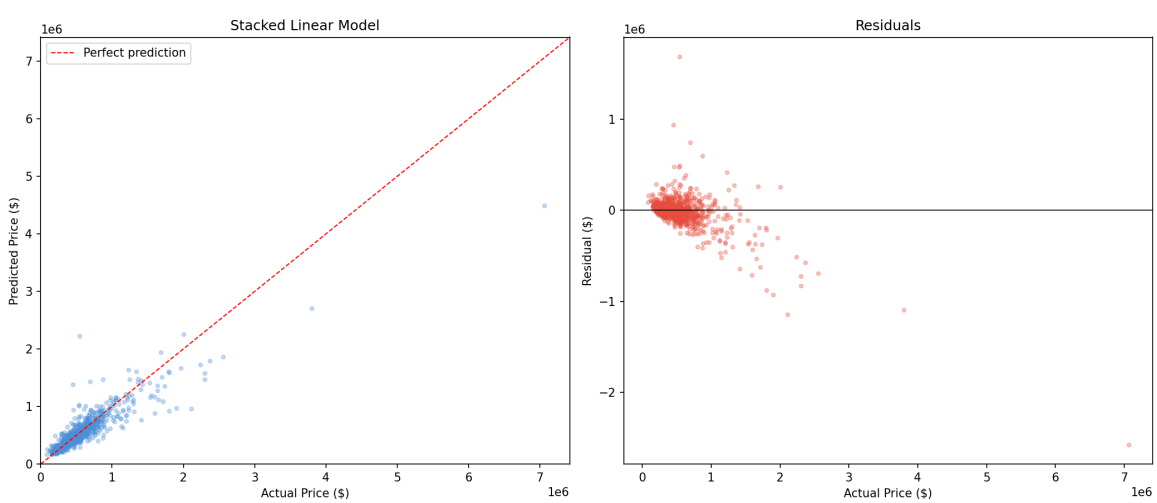

Figure 1: Stacked linear model — predicted vs. actual prices (left) and residuals (right).

The scatter plot demonstrates strong alignment along the perfect-prediction diagonal for most of the price range. The residual plot shows approximately symmetric residuals centered around zero, with increasing variance at higher prices (heteroscedasticity), which is expected and partly mitigated by the log-price target transformation.

7. Model Interpretation

7.1 Feature Importance

Figure 2 shows the top 20 features ranked by absolute standardized coefficient in the best Ridge model. Because all features are standardized, the absolute coefficient magnitude provides a fair comparison of feature influence.

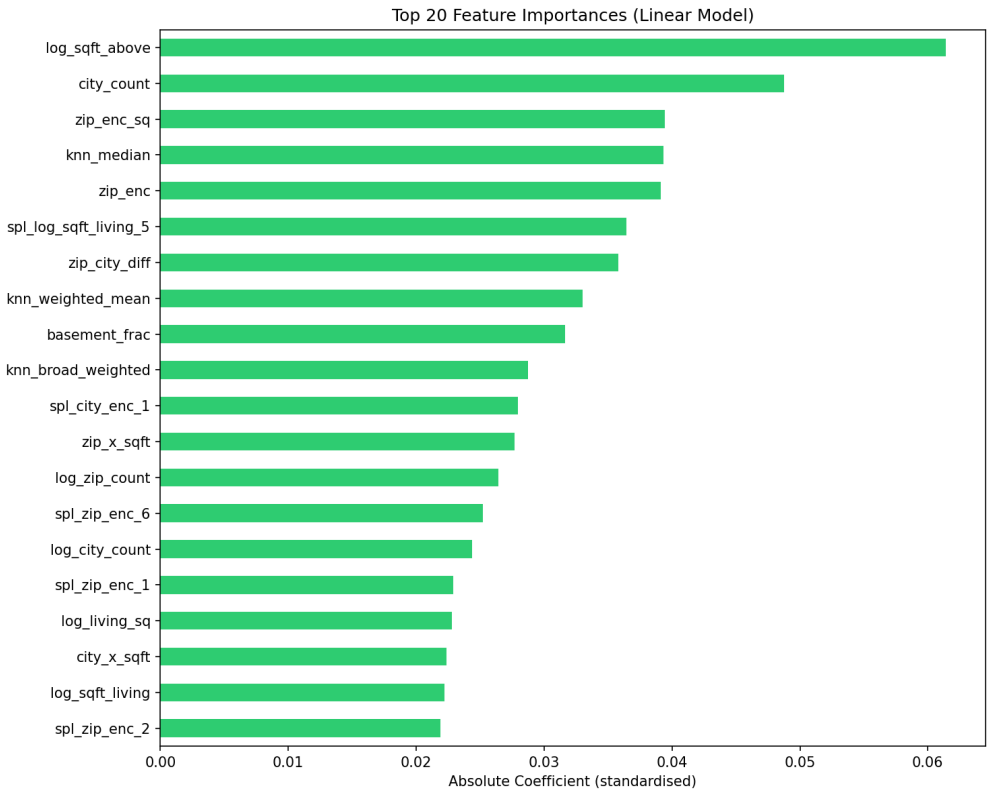

Figure 2: Top 20 features by absolute standardised coefficient in the Ridge model.

7.2 Key Drivers of House Prices

The analysis reveals several important insights about what drives house prices in the Washington state market:

1. Property Size is the Dominant Factor. The log of above-ground living area (log_sqft_above) is the single most influential feature, with a positive coefficient of 0.061. This confirms the intuitive expectation that larger homes command higher prices. The relationship is logarithmic: each percentage increase in size yields a roughly constant percentage increase in price, with diminishing marginal returns.

2. Location Matters Enormously. Location-related features dominate the top ranks: zip code encoding (zip_enc), the squared zip encoding (zip_enc_sq), and the zip-city differential (zip_city_diff) all rank in the top 7. The “city_count” feature (second most important) reflects that properties in more populated, well-established markets tend toward higher prices. The strong contribution of “zip_city_diff” shows that micro-location within a city matters significantly.

3. Neighbourhood Context is Highly Predictive. KNN-derived features rank prominently: “knn_median” (4th), “knn_weighted_mean” (8th), and “knn_broad_weighted” (10th). These features capture the intuition that a house's value is strongly anchored by the prices of comparable nearby properties. The median neighbour price alone carries nearly as much weight as the property's own size features.

4. Basement Proportion Has a Negative Effect. The “basement_frac” feature has a significant negative coefficient (-0.032), indicating that, all else being equal, properties where a larger share of living space is below ground are valued lower. This likely reflects buyer preferences for above-ground living space, natural light, and the lower per-square-foot value attributed to basement areas.

5. Spline Features Capture Non-Linear Patterns. Several spline basis functions for living area (spl_log_sqft_living_5) and location (spl_city_enc_1, spl_zip_enc_6) appear in the top 15, confirming that the relationship between these variables and price is non-linear and that the spline expansion adds meaningful predictive power beyond simple log transforms.

7.3 Practical Implications

For a non-technical client considering deployment of this model, the key takeaways are:

- The model is most reliable for properties priced between $200K and $1M, where it predicts within approximately 15% of the actual price on average.
- Property size and location are the two most important factors. A property's value is largely determined by how big it is and where it is located.
- Comparable sales in the neighbourhood provide strong pricing signals. The model effectively learns from recent sales of similar nearby properties.
- Every coefficient has a clear interpretation: we can explain exactly why the model predicts a given price, which features push the price up, and which push it down. This transparency is a significant advantage over black-box models.

8. Conclusion

This project demonstrates that linear models, when paired with thoughtful and extensive feature engineering, can achieve competitive predictive performance for house price prediction. The stacked linear model achieves a test MAPE of 16.74%, within 1.1 percentage points of a well-tuned XGBoost benchmark (15.64%), while offering full transparency and interpretability.

The feature engineering strategy combines multiple complementary approaches: log transformations to handle skewed distributions, target and count encodings for high-cardinality categorical features, KNN-based neighbourhood features to capture local market dynamics, spline basis expansions for flexible non-linear patterns, KMeans clustering for latent property archetypes, and carefully designed interaction terms to model joint effects. Rigorous cross-validation with feature engineering re-fit inside each fold ensures that all performance estimates are unbiased.

The interpretability of the final model reveals that property size, location, and neighbourhood comparables are the three pillars of house price determination in the Washington state market. These insights are actionable for real estate professionals and align with established domain knowledge, providing both predictions and understanding.